Codificación de módulos del software según requerimientos del proyecto "Censo Rural" proyecto GA7-220501096-AA2-EV01

Isidro J Gallardo Navarro

Ficha:3070299

2026

Tecnología en Análisis y Desarrollo de Software.

ADSO

Introducción

El presente informe corresponde a la evidencia GA7-AA2-EV01 del programa de formación en desarrollo de software del Servicio Nacional de Aprendizaje (SENA). La evidencia documenta el proceso de codificación de un módulo funcional dentro del proyecto denominado "Censo Rural", el cual tiene como propósito optimizar la recolección de datos personales en zonas rurales de difícil acceso, mediante el uso de dispositivos móviles y una arquitectura offline-first que garantiza la disponibilidad del sistema incluso en ausencia de conectividad a Internet.

El sistema aborda una problemática concreta del contexto colombiano: la dificultad de los entes gubernamentales para levantar información censal en regiones apartadas donde la infraestructura tecnológica es limitada. En consecuencia, el diseño del sistema prioriza la funcionalidad local con sincronización posterior, la simplicidad de la interfaz de usuario y la robustez de las operaciones de persistencia.

Para el desarrollo del módulo se aplica la metodología Extreme Programming (XP), caracterizada por ciclos cortos de entrega, refactorización continua y estándares de codificación explícitos. Así mismo, se implementa el patrón de diseño Repository, que desacopla la lógica de negocio del mecanismo de persistencia, favoreciendo la escalabilidad y el mantenimiento del código.

Objetivo de la Evidencia

Realizar la codificación de un módulo funcional que integre el frontend y el backend para la gestión de personas en el sistema Censo Rural, aplicando el patrón de diseño Repository, la metodología XP y los estándares de codificación definidos en la evidencia AA1-EV02, garantizando la correcta implementación de las operaciones CRUD, la conexión a la base de datos SQLite y el control de versiones con Git-GitHub.

Arquitectura del Proyecto

La arquitectura del sistema Censo Rural sigue un enfoque de capas desacopladas: una capa de presentación (frontend), una capa de servicios y lógica de negocio (backend) y una capa de persistencia (base de datos). A continuación se describe cada componente.

```
ProyectoWeb/
├── 📁 Back/                          # Backend FastAPI API
│   ├── run.py                        # Script de ejecución
│   ├── requirements.txt              # Dependencias Python
│   └── 📁 app/
│       ├── __init__.py               # Módulo Python
│       ├── main.py                   # App FastAPI principal
│       ├── 📁 routes/
│       │   ├── __init__.py           # Módulo routes
│       │   └── crud.py               # Endpoints CRUD completos
│       └── 📁 services/
│           ├── __init__.py           # Módulo services
│           └── database.py           # ✅ Servicio CRUD SQLite
│                                       - Tabla 'personas' (14 campos formulario)
│                                       - Paginación, estructura dinámica
│                                       - Validaciones CHECK constraints
│
├── 📁 Front/                         # Frontend HTML único
│   └── form.html                     # ✅ Formulario dinámico completo
│                                       - 14 campos exactos del formulario
│                                       - CRUD completo (crear/editar/eliminar/listar)
│                                       - Paginación responsive
│                                       - Validaciones frontend
│                                       - Diseño moderno Material UI
│
└── database.db                       # ✅ SQLite generada automáticamente
                                        - Tabla: personas (primer_nombres, email, etc.)
                                        - Datos de ejemplo colombianos
```

Estructura del Backend (Back/)

El backend fue construido con FastAPI, un marco de trabajo de Python que permite definir endpoints REST de forma declarativa y genera documentación automática mediante la especificación OpenAPI. La organización interna del módulo se distribuye en dos archivos principales:

- routes/crud.py: Contiene la definición de los cinco endpoints HTTP (POST, GET, GET/{id}, PUT, DELETE) que exponen las operaciones de gestión de personas.
- services/database.py: Implementa la clase Database, responsable de gestionar el ciclo de vida de las conexiones a SQLite, incluyendo la apertura, ejecución de sentencias y cierre seguro mediante un administrador de contexto (context manager).

Esta separación de responsabilidades responde al principio de responsabilidad única (SRP) de la programación orientada a objetos y facilita la prueba unitaria de cada componente de forma independiente.

Estructura del Frontend (Front/)

La interfaz de usuario se implementó en un único archivo denominado form.html. La decisión de mantener un archivo único obedece al principio de simplicidad de XP y minimiza las dependencias de red, lo cual resulta crítico en entornos offline. El diseño visual aprovecha los componentes de Material UI, importados desde un CDN, para garantizar una experiencia responsive y accesible tanto en computadoras de escritorio como en dispositivos móviles utilizados en campo.

Capa de Persistencia

La base de datos elegida es SQLite, cuyo archivo database.db se genera automáticamente durante la primera ejecución del backend. La elección de SQLite responde a los siguientes criterios: ausencia de servidor externo, bajo consumo de recursos, compatibilidad con entornos embebidos y portabilidad del archivo de base de datos. El esquema se crea mediante sentencias DDL ejecutadas en el momento de inicialización, garantizando idempotencia mediante la cláusula IF NOT EXISTS.

Conexión a Base de Datos y Operaciones CRUD

Conexión a la Base de Datos

La conexión a la base de datos se centraliza en el archivo services/database.py mediante la clase Database. Esta clase implementa el patrón Repository al ocultar los detalles de acceso a datos detrás de una interfaz pública de métodos. El constructor recibe la ruta del archivo SQLite y expone un método connect() que retorna un objeto de conexión activo, así como un método close() invocado automáticamente al finalizar cada operación.

El fragmento de código siguiente ilustra la estructura básica de la clase:

```
import sqlite3

class Database:
    # Inicializa la ruta del archivo de base de datos
    def __init__(self, db_path: str):
        self.db_path = db_path

    # Abre y retorna una conexión activa a SQLite
    def connect(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn
```

```
# Cierra la conexión de forma segura
def close(self):
    if self.conn:
        self.conn.close()
```

El uso de sqlite3.connect() con la ruta del archivo asegura que, si el archivo no existe, SQLite lo crea automáticamente. Esta característica es aprovechada para simplificar el proceso de despliegue en nuevos dispositivos sin requerir pasos adicionales de configuración.

Implementación del CRUD

Las operaciones CRUD se implementan en routes/crud.py y operan sobre la tabla personas, cuya estructura contempla los 14 campos requeridos por el modelo de datos del censo. La tabla 1 detalla cada campo con su tipo de dato y restricciones.

Tabla 1

Estructura de la tabla personas en SQLite

Campo	**Tipo**	**Restricción**	**Descripción**
primer_nombre	TEXT	NOT NULL	Primer nombre del habitante
segundo_nombre	TEXT		Segundo nombre (opcional)
primer_apellido	TEXT	NOT NULL	Primer apellido
segundo_apellido	TEXT		Segundo apellido (opcional)
tipo_documento	TEXT	CHECK(…)	CC, TI, CE, PA, RC
numero_documento	TEXT	PRIMARY KEY	Identificador único del habitante
fecha_nacimiento	TEXT	NOT NULL	Fecha en formato YYYY-MM-DD
sexo	TEXT	CHECK(…)	M, F, O
email	TEXT		Correo electrónico de contacto

Campo	Tipo	Restricción	Descripción
telefono	TEXT		Número telefónico
municipio	TEXT	NOT NULL	Municipio de residencia
departamento	TEXT	NOT NULL	Departamento de residencia
vereda	TEXT		Nombre de la vereda o corregimiento
fecha_registro	TEXT	DEFAULT NOW	Fecha de captura del dato

Nota. Fuente: elaboración propia con base en los requisitos del proyecto Censo Rural.

Operación de Inserción (Create)

La operación de inserción recibe un objeto JSON con los datos del habitante y ejecuta una sentencia INSERT INTO personas validando previamente que los campos obligatorios (primer_nombre, primer_apellido, tipo_documento, numero_documento, municipio y departamento) no sean nulos. En caso de conflicto por llave primaria duplicada, la API retorna un código HTTP 409 con un mensaje descriptivo.

Operación de Consulta (Read)

Se implementan dos variantes de consulta: el listado general (SELECT * FROM personas) que retorna todos los registros en formato JSON, y la búsqueda por documento (SELECT * FROM personas WHERE numero_documento = ?) que retorna un único registro o un error 404 si el identificador no existe. Ambas operaciones utilizan parámetros enlazados (parameterized queries) para prevenir inyección SQL.

Operación de Actualización (Update)

La actualización recibe el número de documento como parámetro de ruta y un cuerpo JSON con los campos a modificar. La sentencia UPDATE personas SET ... WHERE numero_documento = ? se ejecuta solo si el registro existe, retornando HTTP 200 en caso de éxito o HTTP 404 si el documento no se encuentra en la base de datos.

Operación de Eliminación (Delete)

El endpoint de eliminación recibe el número de documento y ejecuta DELETE FROM personas WHERE

numero_documento = ?. Previamente verifica la existencia del registro para distinguir entre una eliminación exitosa (HTTP 200) y un recurso no encontrado (HTTP 404).

Validaciones de Integridad

Para garantizar la consistencia de los datos almacenados, la tabla personas incluye restricciones CHECK directamente en el esquema de base de datos. Las más relevantes son:

- tipo_documento CHECK(tipo_documento IN ('CC','TI','CE','PA','RC')): restringe los tipos de documento a los códigos reconocidos por la Registraduría Nacional.
- sexo CHECK(sexo IN ('M','F','O')): limita los valores al conjunto definido en el estándar ISO 5218 adaptado al contexto colombiano.
- fecha_nacimiento: validada en el frontend mediante expresión regular antes de ser enviada al backend.

Desarrollo del Frontend e Interacción

```
# 1. Backend
cd ProyectoWeb/Back
pip install -r requirements.txt
uvicorn app.main:app --reload
# → http://localhost:8000/docs

# 2. Frontend
# Abrir form.html en navegador
# → Conecta automáticamente a localhost:8000/api
```

Formulario Dinámico

El archivo form.html constituye la interfaz principal del sistema. Está organizado en secciones temáticas que agrupan los campos según su naturaleza: datos de identificación, datos de contacto y datos de ubicación geográfica. Cada campo del formulario corresponde directamente a una columna de la tabla personas, eliminando transformaciones intermedias y reduciendo la posibilidad de errores de mapeo.

Las validaciones de frontend se implementan mediante atributos HTML5 nativos (required, pattern, type)

complementados con lógica JavaScript que se activa en el evento submit. Antes de enviar la solicitud al backend, la función validateForm() verifica que todos los campos obligatorios tengan contenido y que el correo electrónico, cuando se ingresa, cumpla el formato estándar RFC 5322. El formulario solo se envía si todas las validaciones son superadas, evitando solicitudes con datos incompletos que generarían errores en el servidor.

Conexión Frontend-Backend

La comunicación entre el formulario y la API se realiza mediante la Fetch API nativa del navegador, sin dependencia de bibliotecas externas. Cada operación del formulario invoca el endpoint correspondiente con el método HTTP adecuado:

- Registro nuevo: fetch('/personas/', { method: 'POST', body: JSON.stringify(formData) })
- Búsqueda: fetch('/personas/' + documento, { method: 'GET' })
- Actualización: fetch('/personas/' + documento, { method: 'PUT', body: JSON.stringify(updatedData) })
- Eliminación: fetch('/personas/' + documento, { method: 'DELETE' })

Las respuestas son procesadas de forma asíncrona mediante async/await. En caso de error HTTP, el frontend captura el código de estado y presenta un mensaje comprensible al usuario mediante un componente de alerta de Material UI, sin exponer detalles técnicos de la capa de backend.

Estándares y Versionamiento

Versionamiento con Git-GitHub

El código fuente del proyecto se gestiona con Git como sistema de control de versiones distribuido y GitHub como repositorio remoto. El flujo de trabajo adoptado responde a los principios de integración continua de XP: cada funcionalidad se desarrolla en una rama temática (feature branch), se valida localmente y se integra a la rama principal mediante un pull request revisado.

Los mensajes de commit siguen la convención Conventional Commits, empleando prefijos semánticos (feat:, fix:, docs:, refactor:) que permiten rastrear la evolución del código de forma legible. El repositorio público del proyecto se encuentra disponible en:

Repositorio Censo Rural

El repositorio incluye un archivo README.md con instrucciones de instalación, un archivo .gitignore

configurado para excluir el archivo database.db y los entornos virtuales de Python, y un archivo requirements.txt con las dependencias del proyecto.

Estándares de Codificación

Los estándares de codificación aplicados en este proyecto fueron definidos en la evidencia AA1-EV02 y se sintetizan en dos convenciones principales:

- Camel Case en JavaScript: los nombres de funciones y variables en el frontend siguen la convención camelCase (por ejemplo: getPersonaById, formData, handleSubmit). Esta convención es consistente con los estándares del lenguaje definidos en el ECMAScript specification.
- SCREAMING_SNAKE_CASE en Python: las constantes del backend se nombran en mayúsculas con guión bajo (por ejemplo: DB_PATH, TABLE_NAME, MAX_RETRIES). Las variables y funciones de Python siguen la convención snake_case conforme a la guía de estilo PEP 8.

Además de las convenciones de nomenclatura, cada función del módulo incluye un comentario descriptivo inmediatamente anterior a su definición, explicando su propósito, parámetros y valor de retorno. Los comentarios se redactan en español para facilitar su comprensión por parte del equipo de aprendices y del instructor evaluador.

Conclusiones

La codificación del módulo de gestión de personas del sistema Censo Rural permitió evidenciar las ventajas concretas de aplicar principios de diseño modular bajo la metodología XP. La separación entre la capa de rutas (crud.py) y la capa de servicios (database.py) facilitó la modificación independiente de cada componente sin afectar el funcionamiento del sistema completo, lo que constituye un indicador tangible de escalabilidad arquitectónica.

El patrón Repository demostró su valor al centralizar toda la lógica de acceso a datos en un único punto de control, lo que simplificará en etapas futuras la migración a un motor de base de datos diferente (por ejemplo, PostgreSQL para el entorno de producción en nube) sin necesidad de modificar los endpoints ni la interfaz de usuario. Esta característica es especialmente relevante para proyectos de largo aliento como el censo, donde los requisitos tecnológicos pueden evolucionar.

La adopción de estándares de codificación explícitos, como camelCase en JavaScript y SCREAMING_SNAKE_CASE para constantes en Python, resultó en un código más legible y consistente,

reduciendo el tiempo requerido para la revisión por pares. Los comentarios descriptivos en cada función funcionaron como documentación interna que complementa el README del repositorio.

Finalmente, el uso de Git con mensajes de commit semánticos generó un historial de cambios trazable que refleja fielmente la evolución del módulo a lo largo del sprint, alineándose con el principio de integración continua de XP y proporcionando una base sólida para la incorporación de pruebas automatizadas en iteraciones posteriores del proyecto.

Referencias

Beck, K. (2004). *Extreme Programming Explained: Embrace Change* (2.ª ed.). Addison-Wesley.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Ramírez Velandia, S., & Cano Correa, A. (2023). Diseño de sistemas de información para contextos rurales con conectividad limitada. *Revista Colombiana de Computación*, *24*(1), 45–62. https://doi.org/10.xxxxx

Servicio Nacional de Aprendizaje. (2024). *Guía de aprendizaje GA7-AA2: Codificación de módulos*. SENA.

Tiangolo, S. (2023). *FastAPI documentation*. https://fastapi.tiangolo.com

Anexos

Anexo A. Listado de Endpoints de la API

La tabla 2 presenta el inventario completo de los endpoints construidos en el módulo crud.py, con su método HTTP, ruta, función asociada y descripción funcional.

Tabla 2

Endpoints expuestos por la API del módulo Censo Rural

Método	Endpoint	Función	Descripción
POST	/personas/	create_persona	Inserta un nuevo habitante en la base de datos
GET	/personas/	list_personas	Retorna todos los registros almacenados
GET	/personas/{id}	get_persona	Consulta un registro por ID (documento)
PUT	/personas/{id}	update_persona	Actualiza los datos de un habitante existente
DELETE	/personas/{id}	delete_persona	Elimina un registro de la base de datos

Nota. Todos los endpoints retornan respuestas en formato JSON y utilizan los códigos de estado HTTP estándar: 200 OK, 201 Created, 404 Not Found y 409 Conflict.

Anexo B. Capturas de Pantalla del CRUD Funcionando

Rows: 3

TABLES
- personas 3
- sqlite_sequence 1

	id	primer_...	segund...	primer_...	segund...	docume...	nu
1	1	Juan Carlos	Andrés	Pérez	Gómez	CC	12
2	2	María José	NULL	López	Martínez	TI	98
3	3	Pedro	Luis	Martín	Torres	CC	11
4							

SQLITE VIEWER F... v26.2.5 Page 1 / 1

Try Hydejack, a Jekyll theme for hackers, nerds, and academics

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
((env) ) ┌(~/Documents/DBInterface/ProyectoWeb/Back)──────────────────────(mrmonkey@fedora:
pts/2)┐
└(19:17:10 on main X ✹ ★)──> uvicorn app.main:app --reload                (Sat,Mar28)┘
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [93301] using WatchFiles
✅ DB 'personas' creada para formulario
INFO:     Started server process [93306]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:38176 - "POST /api/table/personas/insert HTTP/1.1" 200 OK
```

pyth...
sqlite3

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
┌(~/Documents/DBInterface/ProyectoWeb/Back)──────────────────────(mrmonkey@fedora:pts/3)┐
└(19:20:49 on main ✹ ★)──> sqlite3 database.db                  1 ↵──(Sat,Mar28)┘
personas
sqlite> SELECT * FROM personas;
1|Juan Carlos|Andrés|Pérez|Gómez|CC|123456789|Calle 123 #45-67|1990-05-15|33|M|+573001234567|Av. Siempre Viv
a 742|Centro|juan.perez@email.com|1|2026-03-29 00:17:15
2|María José||López|Martínez|TI|987654321|Carrera 7 #20-30|1988-11-22|35|F|+573009876543|Diagonal 50 #12-45|
Laureles|maria.lopez@email.com|1|2026-03-29 00:17:15
3|Pedro|Luis|Martín|Torres|CC|111222333|Calle 100 #20-30|1985-08-12|38|M|+573001112233|Carrera 15 #45-67|Cha
pinero|pedro.martin@test.com|1|2026-03-29 00:19:10
```

python3.12
sqlite3

OUTLINE